

**MODEL SUDOKU SOLVER PENDEKATAN
GREEDY
MENUJU DYNAMIC PROGRAMING**

Pengembangan Tahap 1

Disusun oleh:

SekarSendiriSaja

2026

ABSTRAK

Abstrak

Laporan ini mendokumentasikan tahapan pengembangan program solver Sudoku yang ditulis dalam bahasa pemrograman Go. Pengembangan dilakukan secara bertahap, dimulai dari pemodelan aturan permainan, validasi papan, hingga pencarian kandidat angka untuk setiap sel kosong. Pendekatan awal menggunakan strategi greedy berbasis aturan eliminasi, dengan rencana pengembangan lanjutan ke arah dynamic programming. Dokumen ini memuat landasan teori, pemetaan struktur data, formula iterasi subgrid, serta catatan teknis dari setiap fase yang telah diselesaikan.

Kata kunci: Sudoku, solver, Go, greedy, dynamic programming, kandidat, eliminasi.

DAFTAR ISI

I. Pendahuluan	3
II. Landasan Teori Aturan Sudoku	3
III. Pemetaan Posisi dalam Subgrid 3×3	5
IV. Checkpoint Progres Pengembangan	6
IV.1 Bagian I Pemodelan dan Validasi	7
IV.2 Bagian II Pencarian Kandidat	8
V. Catatan Pengembangan dan Ide Lanjutan	10
VI. Kesimpulan	11
VII. Analisis Kompleksitas dan Optimalitas Bagian II	12
VII.1 Kompleksitas Waktu	12
VII.2 Kompleksitas Ruang	13
VII.3 Apakah Sudah Optimal?	13

I. PENDAHULUAN

Sudoku merupakan teka-teki logika berbasis angka yang telah menjadi salah satu permasalahan klasik dalam bidang kecerdasan buatan dan pemrograman kompetitif. Permasalahan ini menarik untuk dikaji karena memiliki ruang pencarian yang besar namun dapat diselesaikan secara efisien melalui teknik deduksi logis terstruktur.

Pengembangan program solver Sudoku pada proyek ini dilakukan secara iteratif. Setiap fase pembangunan dimulai dengan pemahaman aturan domain yang kemudian diimplementasikan secara langsung ke dalam kode Go tanpa penggunaan pustaka eksternal. Pendekatan ini bertujuan agar setiap keputusan implementasi dapat dipertanggungjawabkan secara algoritmis.

Dokumen ini merupakan catatan resmi dari progres yang telah dicapai, disusun berdasarkan catatan kerja yang terdapat dalam lembar kerja pengembangan (*Buku1.xlsx*). Laporan mencakup dua fase utama yang telah diselesaikan dan satu fase yang masih dalam tahap perencanaan.

II. LANDASAN TEORI ATURAN SUDOKU

II.1 Struktur Papan

Papan Sudoku terdiri dari 9 baris dan 9 kolom, membentuk total 81 sel. Sel-sel tersebut dikelompokkan ke dalam 9 kotak (blok) berukuran 3×3 yang dibatasi oleh garis tegas. Setiap sel memiliki koordinat (i, j) dengan i menyatakan baris dan j menyatakan kolom, keduanya bernilai antara 1 sampai 9.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Gambar 1. Contoh papan Sudoku 9×9 dengan angka petunjuk awal (given) ditampilkan dengan latar abu-abu.

II.2 Aturan Permainan

Terdapat tiga aturan fundamental yang harus dipenuhi secara simultan:

Aturan Baris. Setiap baris harus memuat angka 1, 2, 3, 4, 5, 6, 7, 8, dan 9 masing-masing tepat satu kali. Tidak diperbolehkan adanya pengulangan angka dalam satu baris yang sama.

Aturan Kolom. Setiap kolom harus memuat angka 1 sampai 9 tepat satu kali. Tidak diperbolehkan adanya pengulangan angka dalam satu kolom yang sama.

Aturan Kotak 3×3 . Setiap kotak berukuran 3×3 (subgrid) harus memuat angka 1 sampai 9 tepat satu kali. Tidak diperbolehkan adanya angka yang sama dalam satu kotak yang sama.

II.3 Ketentuan Tambahan

Angka Petunjuk Awal (*Clue*). Sebagian sel telah diisi sejak awal sebagai titik acuan. Angka-angka ini bersifat tetap dan tidak boleh diubah atau dihapus selama proses penyelesaian.

Keunikan Solusi. Setiap teka-teki Sudoku yang valid hanya memiliki tepat satu solusi. Pemain atau program harus menemukan solusi tersebut melalui proses deduksi logis.

Larangan Menebak. Penyelesaian harus dilakukan dengan deduksi logis murni. Setiap pengisian angka harus dapat dibuktikan secara logis berdasarkan kondisi papan yang ada, bukan berdasarkan coba-coba.

Batasan Angka. Hanya angka 1 hingga 9 yang digunakan. Angka 0, bilangan negatif, maupun huruf tidak diperkenankan hadir di dalam papan permainan.

III. PEMETAAN POSISI DALAM SUBGRID 3×3

Dalam proses pengembangan, setiap sel di dalam subgrid 3×3 diberi label posisi relatif menggunakan huruf alfabet. Pemetaan ini disebut sebagai **MAT-RULE** dan berfungsi sebagai kunci identifikasi posisi (*key mapping*) saat melakukan operasi iterasi pada blok.

III.1 Skema Penamaan Posisi dalam Satu Blok

Setiap blok 3×3 memiliki sembilan sel yang diberi nama sebagai berikut:

<i>a</i>	<i>d</i>	<i>g</i>
<i>b</i>	<i>e</i>	<i>h</i>
<i>c</i>	<i>f</i>	<i>i</i>

Gambar 2. Penamaan posisi relatif dalam satu blok 3×3 (MAT-RULE).

III.2 Distribusi Label pada Seluruh Papan

Pola penamaan di atas diulang secara konsisten untuk setiap blok pada seluruh papan 9×9. Tabel berikut menunjukkan distribusi label posisi untuk setiap koordinat baris dan kolom:

Tabel 1. Peta posisi relatif (label) untuk setiap sel pada papan 9×9.

Baris \ Kolom	1	2	3	4	5	6	7	8	9
1	a	d	g	a	d	g	a	d	g
2	b	e	h	b	e	h	b	e	h
3	c	f	i	c	f	i	c	f	i
4	a	d	g	a	d	g	a	d	g
5	b	e	h	b	e	h	b	e	h
6	c	f	i	c	f	i	c	f	i
7	a	d	g	a	d	g	a	d	g
8	b	e	h	b	e	h	b	e	h
9	c	f	i	c	f	i	c	f	i

Pola ini bersifat periodik dengan periode 3 pada kedua dimensi, mencerminkan struktur blok 3×3 yang berulang pada papan Sudoku. Label $a-i$ berfungsi sebagai indeks relatif posisi suatu sel di dalam bloknya, terlepas dari koordinat absolutnya pada papan.

IV. CHECKPOINT PROGRES PENGEMBANGAN

Pengembangan program mengikuti alur bertahap yang direncanakan dari pendekatan *greedy* menuju *dynamic programming*. Tabel berikut merangkum seluruh poin pencapaian yang telah dicatat.

Tabel 2. Checkpoint progres pengembangan program Sudoku solver.

No.	Deskripsi Pencapaian	Status
1	Membuat aturan dasar, pemetaan kunci, dan konsep dasar program (<i>Make a Rule, Key Mapping, Base Concept</i>) membangun model program awal	Selesai
2	Menghitung catatan kandidat dalam baris dan kolom (<i>Pencarian Kandidat</i>) implementasi eliminasi berbasis tiga aturan dengan formula iterasi subgrid	Selesai
3	Memeriksa kondisi posisi yang kosong berdasarkan kombinasi relatif baris dan kolom menggunakan struct sebagai identitas koordinat	Direncanakan

IV.1 Bagian I Pemodelan Papan dan Validasi

Pada fase pertama, program dibangun untuk membaca input papan Sudoku secara interaktif dan memvalidasi kondisi awal papan terhadap aturan baris dan kolom. Representasi papan menggunakan array dua dimensi bertipe integer dengan ukuran 10×10 , di mana indeks baris dan kolom dimulai dari 1.

Format Input

Setiap token input terdiri dari tiga karakter yang masing-masing mewakili koordinat baris (x), koordinat kolom (y), dan nilai sel (v). Input diakhiri dengan token khusus **000** sebagai penanda terminasi.

```
// Struktur representasi papan
var sudo = [10][10]int{ ... }
// indeks 0 diabaikan, valid: 1-9

// Pembacaan input
var a string
for true {
    fmt.Scan(&a)
    if a == "000" { break }
    sudo[(a[0]-48)][(a[1]-48)] = int(a[2] - 48)
}
```

Validasi Baris dan Kolom

Validasi dilakukan dengan menghitung frekuensi kemunculan setiap angka pada setiap baris dan kolom. Jika ada angka yang muncul lebih dari satu kali, papan dinyatakan tidak valid. Pendekatan ini menggunakan array cacah (*count unique value*) yang direset setelah setiap iterasi.

```

var janggal bool = false
var count [10]int
for i := 1; i <= 9; i++ {
    for j :=1; j <=9; j++ {
        count[sudo[i][j]]++
    }
    for k :=1; k <=9; k++ {
        if count[k]> 1 {
            janggal = true
        }
    }
    count = [10]int{}
    for i := 1; i <= 9; i++ {
        for j :=1; j <=9; j++ {
            count[sudo[j][i]]++
        }
        for k :=1; k <=9; k++ {
            if count[k]> 1 {
                janggal = true
            }
        }
    }
    count = [10]int{}
}
if janggal {
    fmt.Println("sudoku tidak valid")
} else {
    fmt.Println("sudoku valid")
}

```

Perlu dicatat bahwa pada fase ini, validasi kotak 3×3 belum diimplementasikan secara eksplisit karena adanya keterbatasan dalam merangkum kondisi blok ke dalam satu ekspresi kondisional sederhana.

IV.2 Bagian II Pencarian Kandidat

Setelah papan dinyatakan valid, program melanjutkan ke fase analisis kandidat. Pada fase ini, setiap sel kosong ($sudok[i][j] == 0$) dianalisis untuk menentukan himpunan angka yang secara legal dapat ditempatkan di sel tersebut. Proses ini merupakan inti dari teknik eliminasi berbasis aturan (*constraint propagation*).

Struktur Data Kandidat

Untuk merepresentasikan kondisi setiap sel beserta informasi kandidatnya, digunakan tipe data *struct* dengan sembilan medan angka dan satu medan pencacah total kandidat.

```
type candidate struct {  
    a1, a2, a3, a4, a5, a6, a7, a8, a9 int  
    totalcandidate int  
}  
var sudocek [10][10]candidate
```

Sel yang telah terisi diberikan nilai $totalcandidate = 10$ sebagai penanda bahwa sel tersebut bukan merupakan kandidat aktif dan tidak perlu diproses lebih lanjut.

Mekanisme Eliminasi Tiga Arah

Proses eliminasi kandidat dilakukan terhadap tiga domain secara berurutan untuk setiap sel kosong: baris, kolom, dan subgrid 3×3 . Array sementara *b1* diinisialisasi dengan nilai 1–9, kemudian posisi yang telah terpakai di ketiga domain di-nol-kan. Angka yang tersisa merupakan kandidat yang valid.

```
b1 := [10]int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

// Eliminasi berdasarkan baris ke-i
for i2 := 1; i2 <= 9; i2++ {
    b1[sudo[i][i2]]=0
}
// Eliminasi berdasarkan kolom ke-j
for j2 :=1; j2 <=9; j2++ {
    b1[sudo[j2][j]]=0 }
// Eliminasi berdasarkan subgrid 3x3
for i2 :=(i-1)/3*3 + 1; i2 <=(i-1)/3*3+3; i2++ {
    for j2 :=(j-1)/3*3 + 1; j2 <=(j-1)/3*3+3; j2++ {
        b1[sudo[i2][j2]]=0
    }
}
```

Formula Iterasi Subgrid

Formula kunci yang digunakan untuk menentukan batas awal dan batas akhir iterasi pada subgrid 3×3 yang memuat sel (i, j) adalah:

$$\begin{aligned} i_{\text{start}} &= \lfloor (i-1)/3 \rfloor \times 3 + 1 & i_{\text{end}} &= \lfloor (i-1)/3 \rfloor \times 3 + 3 \\ j_{\text{start}} &= \lfloor (j-1)/3 \rfloor \times 3 + 1 & j_{\text{end}} &= \lfloor (j-1)/3 \rfloor \times 3 + 3 \end{aligned}$$

Formula ini memanfaatkan pembagian bilangan bulat (integer division) untuk mengidentifikasi blok mana yang memuat sel dengan koordinat (i, j) , kemudian menghitung batas bawah dan batas atas iterasi blok tersebut secara otomatis.

Tabel 3. Hasil kalkulasi batas iterasi subgrid untuk nilai $i = 1$ hingga 9.

i	$\lfloor (i-1)/3 \rfloor$	i_{start}	i_{end}	Blok
1	0	1	3	Blok atas
2	0	1	3	Blok atas
3	0	1	3	Blok atas
4	1	4	6	Blok tengah
5	1	4	6	Blok tengah
6	1	4	6	Blok tengah
7	2	7	9	Blok bawah
8	2	7	9	Blok bawah
9	2	7	9	Blok bawah

V. CATATAN PENGEMBANGAN DAN IDE LANJUTAN

V.1 Ide Tahap Berikutnya

Berdasarkan catatan pada Lembar 3 (*Lembar3*) dari dokumen pengembangan, ide utama yang akan diimplementasikan pada fase selanjutnya adalah pemeriksaan kondisi posisi sel yang kosong berdasarkan kombinasi relatif baris dan kolom. Identitas koordinat setiap sel akan direpresentasikan menggunakan *struct* tersendiri untuk memudahkan manipulasi dan pelacakan status sel selama proses penyelesaian.

Secara konseptual, langkah ini merupakan transisi dari pendekatan pencarian kandidat yang bersifat lokal (per sel) menuju analisis global yang mempertimbangkan hubungan antar sel dalam satu baris, kolom, atau blok.

V.2 Strategi Lanjutan yang Direncanakan

Tabel 4. Strategi penyelesaian yang direncanakan untuk fase berikutnya.

Strategi	Deskripsi	Pendekatan
<i>Naked Single</i>	Jika suatu sel hanya memiliki tepat satu kandidat, nilai tersebut langsung diisikan tanpa perlu analisis lebih lanjut.	Greedy
<i>Hidden Single</i>	Jika suatu angka hanya dapat ditempatkan di satu posisi dalam sebuah baris, kolom, atau blok, maka angka tersebut diisikan pada posisi itu.	Greedy
<i>Backtracking</i>	Untuk kasus di mana deduksi sederhana tidak cukup, program akan mencoba pengisian secara sistematis dan mundur (<i>backtrack</i>) jika terjadi kontradiksi.	Dynamic Programming

V.3 Catatan Teknis Implementasi

Beberapa kendala teknis yang dicatat selama proses pengembangan antara lain:

- Validasi blok 3×3 tidak dapat menggunakan fungsi tunggal berbasis *sum* $== 45$ karena kondisi tersebut tidak dapat membedakan antara blok yang valid dan blok yang masih memiliki sel kosong dengan distribusi angka yang menyebabkan jumlah sama dengan 45.
- Untuk baris dan kolom, validasi menggunakan pendekatan *count unique value* yang lebih andal dan tidak bergantung pada asumsi nilai total.
- Indeks papan dimulai dari 1 (bukan 0) agar pemetaan koordinat lebih natural dan sesuai dengan konvensi Sudoku. Indeks ke-0 pada array dibiarkan tidak digunakan.
- Ide teknis yang dicatat untuk iterasi sel: gunakan operasi modulo dan pembagian untuk mengekstrak dan mengkombinasikan angka digit dari representasi berbasis 10 (*mod 10 untuk collect, div 10 untuk eliminate*).

VI. KESIMPULAN

Program solver Sudoku ini telah berhasil melewati dua fase pengembangan awal. Fase pertama menghasilkan model papan yang fungsional beserta validasi baris dan kolom yang bekerja secara benar. Fase kedua menambahkan kemampuan pencarian kandidat pada setiap sel kosong melalui eliminasi tiga arah yang mencakup baris, kolom, dan subgrid 3×3 .

Formula iterasi subgrid yang ditemukan, yaitu $\lfloor (i-1)/3 \rfloor \times 3 + 1$ hingga $\lfloor (i-1)/3 \rfloor \times 3 + 3$, merupakan kontribusi inti dari fase ini karena memungkinkan program mengidentifikasi batas blok secara dinamis tanpa memerlukan tabel pencarian atau kondisi percabangan eksplisit.

Pengembangan berikutnya akan berfokus pada pemanfaatan data kandidat yang telah dikumpulkan untuk mengimplementasikan strategi deduksi tingkat lanjut seperti *naked single* dan *hidden single*, dilanjutkan dengan transisi ke pendekatan dynamic programming untuk menangani kasus-kasus yang tidak dapat diselesaikan melalui deduksi sederhana.

VII. ANALISIS KOMPLEKSITAS DAN OPTIMALITAS

BAGIAN II

Bagian ini menyajikan analisis formal terhadap efisiensi program *bagian2.go*, mencakup kompleksitas waktu dan kompleksitas ruang, serta penilaian apakah implementasi yang ada telah mencapai batas optimalitas untuk permasalahan pencarian kandidat pada papan Sudoku.

VII.1 Kompleksitas Waktu

Program terdiri dari beberapa fase yang dieksekusi secara sekuensial. Karena ukuran papan Sudoku bersifat tetap ($N = 9$), seluruh nilai konstan dapat dihitung secara eksak. Namun, analisis dilakukan secara generik terhadap N agar pola algoritmik dapat dipahami secara universal.

Tabel 5. Rincian kompleksitas waktu per fase program *bagian2.go*.

Fase	Loop / Operasi	Kompleksitas	Ops nyata (N=9)
Baca input	Loop hingga token 000; K token clue dibaca	$O(K)$	≤ 81
Cetak papan	Loop $i=1..9, j=1..9$	$O(N^2)$	81
Validasi baris	Loop $i \times (j + k) = 9 \times (9+9)$	$O(N^2)$	162
Validasi kolom	Loop $i \times (j + k) = 9 \times (9+9)$	$O(N^2)$	162
Analisis kandidat eliminasi baris	Loop $i \times j \times i2 = 9 \times 9 \times 9$	$O(N^3)$	729
Analisis kandidat eliminasi kolom	Loop $i \times j \times j2 = 9 \times 9 \times 9$	$O(N^3)$	729
Analisis kandidat eliminasi kotak 3×3	Loop $i \times j \times i2 \times j2 = 9 \times 9 \times 3 \times 3$	$O(N^3)$	729
Scan array <i>bl</i> (hitung kandidat)	Loop $i \times j \times k = 9 \times 9 \times 10$	$O(N^3)$	810
Cetak totalcandidate	Loop $i \times j = 9 \times 9$	$O(N^2)$	81

Fase dengan kompleksitas tertinggi adalah fase analisis kandidat yang terdiri atas empat sub-loop bersarang. Total operasi pada fase ini mencapai $729 + 729 + 729 + 810 = 2.997$ iterasi untuk $N = 9$. Ditambah dengan fase sebelumnya, total operasi keseluruhan program adalah sekitar **3.645 operasi dasar**.

$$\begin{aligned}
 T(N) &= O(N^2) + O(N^2) + O(N^2) + O(N^3) + O(N^3) + \\
 &\quad O(N^3) + O(N^3) \\
 &= O(N^3) \quad (\text{didominasi oleh fase analisis} \\
 &\quad \text{kandidat})
 \end{aligned}$$

Untuk $N = 9$ (papan Sudoku standar), program ini selalu berjalan dalam waktu yang sepenuhnya konstan karena N tidak pernah berubah. Namun secara algoritmik, pola kompleksitas waktunya adalah **$O(N^3)$** kubik terhadap ukuran sisi papan.

VII.2 Kompleksitas Ruang

Analisis kompleksitas ruang mencakup seluruh variabel yang dialokasikan secara statis maupun dinamis selama eksekusi program. Semua alokasi bersifat statis (tidak ada penggunaan heap atau rekursi) sehingga penggunaan memori dapat dihitung secara tepat.

Tabel 6. Rincian kompleksitas ruang per struktur data *bagian2.go*.

Variabel	Deskripsi	Ukuran	Kompleksitas
<code>sudo [10][10]int</code>	Papan Sudoku utama (1-indexed)	100 int	$O(N^2)$
<code>sudoccek [10][10]candidate</code>	Array struct kandidat; tiap struct = 10 field int	1.000 int	$O(10 \cdot N^2) = O(N^2)$
<code>count [10]int</code>	Penghitung frekuensi, direset setiap iterasi	10 int	$O(N)$
<code>b1 [10]int</code>	Buffer kandidat sementara, dibuat ulang tiap sel; hanya satu instans aktif pada satu waktu	10 int	$O(N)$
<code>a string</code>	Token input 3 karakter	3 byte	$O(1)$
<code>janggal bool</code>	Flag validasi papan	1 byte	$O(1)$

$$S(N) = O(N^2) + O(N^2) + O(N) + O(N) + O(1) = O(N^2) \quad (\text{didominasi oleh } sudo \text{ dan } sudoccek)$$

Dalam satuan byte (asumsi `int` = 8 byte pada Go 64-bit): total memori $\approx (100 + 1.000) \times 8 \text{ byte} = \mathbf{8.800 \text{ byte} \approx 8,6 \text{ KB}}$. Ini merupakan alokasi yang sangat kecil dan sepenuhnya berada dalam cache L1 prosesor modern.

VII.3 Apakah Sudah Optimal?

Setelah menghitung kompleksitas formal, pertanyaan berikutnya adalah apakah implementasi ini telah mencapai batas optimalitas untuk masalah pencarian kandidat Sudoku. Analisis dilakukan dari tiga sisi: *lower bound* teoritis, redundansi perhitungan, dan efisiensi struktur data.

Lower Bound Teoritis

Untuk menentukan kandidat suatu sel kosong, program *wajib* memeriksa paling sedikit 20 sel yang berada dalam satu baris (9 sel), satu kolom (9 sel), dan satu kotak 3×3 (9 sel), dikurangi tumpang tindih (maksimum 3 sel). Dengan demikian, batas bawah untuk memproses satu sel adalah $O(20) = O(N)$, dan untuk seluruh 81 sel adalah $O(N^3)$ identik dengan kompleksitas yang dicapai program saat ini.

Lower bound pencarian kandidat = $O(N^3)$ Kompleksitas aktual program = $O(N^3)$ → Program sudah optimal secara asimtotik.
--

Analisis Redundansi dan Inefisiensi Konstan

Meskipun secara asimtotik sudah optimal ($O(N^3)$), terdapat tiga inefisiensi pada faktor konstanta yang dapat diperbaiki tanpa mengubah kelas kompleksitas.

Tabel 7. Daftar inefisiensi konstan dan rekomendasi perbaikan.

No.	Inefisiensi	Penjelasan	Dampak
1	Eliminasi kotak 3×3 menghitung ulang sel yang sudah di-scan baris/kolom	Sel sudut kotak (yang berada dalam baris atau kolom yang sama dengan sel target) di-nolkan dua kali: sekali saat eliminasi baris/kolom, sekali lagi saat eliminasi kotak. Tidak mengubah hasil karena $b1[x] = 0$ bersifat idempoten, tetapi menghasilkan operasi tulis yang sia-sia.	Redundansi ≤ 6 tulis/sel
2	Field $a1-a9$ dalam struct tidak pernah diisi	Struct candidate memiliki 10 field, namun hanya totalcandidate yang diperbarui. Sembilan field $a1-a9$ dialokasikan tetapi tidak dimanfaatkan, menyebabkan pemborosan ruang sebesar 90% dari ukuran struct.	+900 int ruang tidak terpakai
3	Sel terisi tetap masuk ke loop kandidat sebelum dicek	Pemeriksaan <code>if sudo[i][j] == 0</code> dilakukan di dalam loop terdalam. Sel yang sudah terisi tetap menghabiskan satu siklus iterasi sebelum diskip. Idealnya kondisi ini dicek lebih awal (di level loop luar) untuk menghindari overhead branch.	Overhead per sel terisi

Rekomendasi Perbaikan

Berdasarkan analisis di atas, dua perbaikan berikut dapat meningkatkan efisiensi konstan program tanpa mengubah kompleksitas asimtotik:

1. **Sederhanakan struct** ganti candidate dengan type candidate
`struct { mask uint16; totalcandidate int }`, di mana satu bitfield 9-bit merepresentasikan kandidat 1–9. Ini mengurangi ukuran struct dari 80 byte menjadi 16 byte (penghematan 80%).
2. **Pisahkan loop sel kosong dan sel terisi** tambahkan kondisi `continue` di awal loop sebelum alokasi `b1`, sehingga sel terisi sama sekali tidak menghabiskan alokasi array.

Kesimpulan Optimalitas

Tabel 8. Ringkasan kompleksitas dan status optimalitas *bagian2.go*.

Dimensi	Nilai (Generik)	Nilai (N=9)	Status
Kompleksitas Waktu	$O(N^3)$	≈ 3.645 ops	Asimtotik optimal
Kompleksitas Ruang	$O(N^2)$	$\approx 8,6$ KB	Ada pemborosan (90% struct tidak terpakai)
Faktor Konstanta Waktu		Ada redundansi ≤ 6 tulis/sel	Dapat diperbaiki
Correctness		Logika eliminasi benar	Benar dan lengkap